PRAKTIKUM 4
MENGIRIM PESAN KE EMAIL DARI BOARD ESP32 MENGGUNAKAN SMTP SERVER HTML, TEXT, DAN ATTACHMENTS (ARDUINO IDE)

NRP	: 1. 3223600046
	2. 3223600047
	3. 3223600056
	4. 3223600058
	5. 3223600060
Nama	: 1. Hari Tammim Mu'izha
	2. Bintang Endra Erlangga
	3. Mochamad Najib Thariq
	4. Ihsanta Zaki Sanjaya
	5. Mochammad Ega Yudhistira
Kelas	: 2 Teknik Komputer B
Mata Kuliah	: Praktikum Alat Pengembangan Perangkat Lunak
Tanggal	: Kamis, 13 Maret 2025
Dosen	: Dr. Setiawardhana, S.T., M.T.

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

MENGIRIM PESAN KE EMAIL DARI BOARD ESP32 MENGGUNAKAN SMTP SERVER HTML, TEXT, DAN ATTACHMENTS (ARDUINO IDE)

A. Dasar Teori

DESKRIPSI

Di sini kita akan belajar bagaimana NodeMCU ESP32 mengirim pesan ke email menggunakan Server SMPT. Pengiriman pesan tidak hanya berupa teks, tapi juga bisa melampirkan gambar dan file (.txt). Pemrograman ESP32 menggunakan Arduino IDE.

Gambar 1. Pengiriman pesan ke email menggunakan NodeMCU

INTRODUCTION SERVER SMTP

SMTP berarti *Simple Mail Transfer Protocol* dan merupakan standar internet untuk transmisi email. Untuk mengirim email menggunakan ESP32, Anda perlu menghubungkannya ke Server SMTP.

ESP32 MAILCLIENT LIBRARY

Untuk mengirim email dengan ESP32, kita akan menggunakan *MailClient ESP32 library*. Library ini digunakan untuk mengirim dan menerima email dengan atau tanpa lampiran melalui server SMTP dan IMAP.

Dalam praktikum ini kita akan menggunakan SMTP untuk mengirim email dengan dan tanpa lampiran. Sebagai contoh, kami akan mengirimkan file gambar (.jpg) dan teks (.txt). File yang akan dikirim melalui email dapat disimpan di ***ESP32 SPIFFS*** atau kartu ***SD card***.

B. Alat dan Bahan

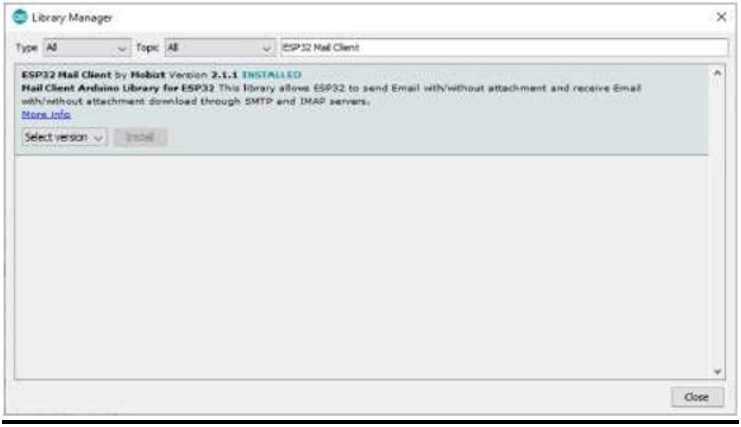
1. ESP32
2. Kabel USB Micro
3. Arduino IDE

C. Prosedur

INSTALLING THE ESP32 MAILCLIENT LIBRARY

Sebelum melanjutkan, Anda perlu menginstal **ESP32 MailClient Libray**. Library ini dapat diinstal melalui Arduino IDE Library Manager. Di Arduino IDE, buka **Sketch> Include Library> Manage Libraries...**

Ketika Library Manager terbuka. Cari **ESP32 Mail Client** by Mobizt dan install.

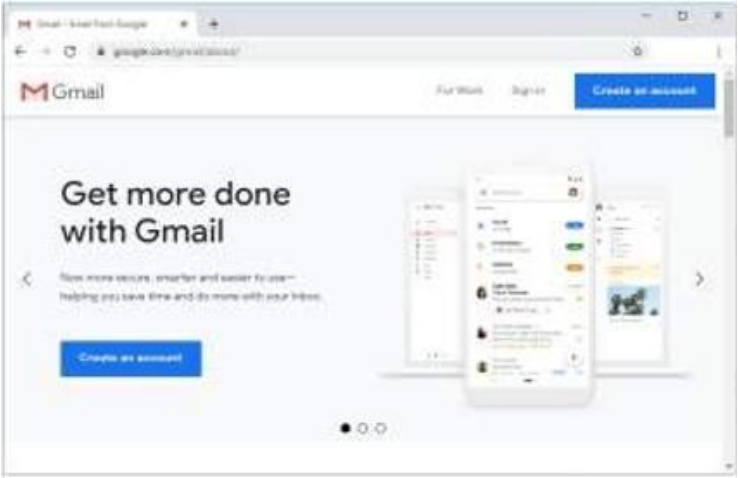


SENDER EMAIL (NEW ACCOUNT)

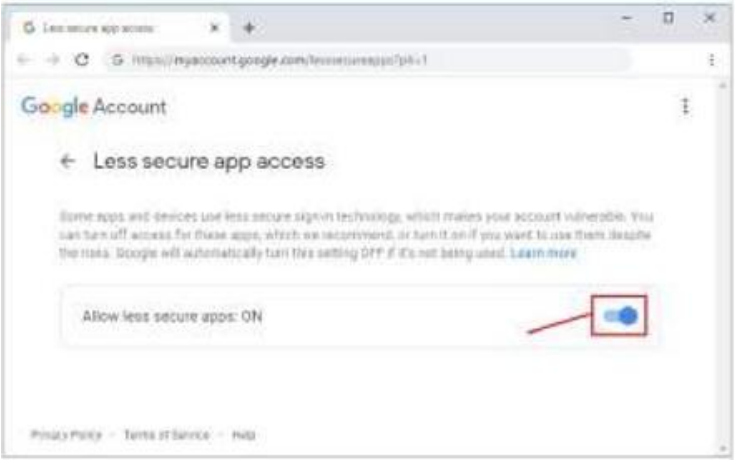
DIREKOMENDASIKAN MEMBUAT AKUN EMAIL BARU untuk mengirim email. Jangan gunakan email pribadi utama Anda untuk mengirim email melalui ESP32. Jika terjadi kesalahan pada kode Anda atau jika karena kesalahan Anda membuat terlalu banyak permintaan, Anda dapat diblokir atau akun Anda dinonaktifkan sementara.

CREATE A SENDER EMAIL ACCOUNT

1. Buat akun di GMAIL atau penyedia email lainnya



2. Pilih > ***Allow less secure apps : ON***



3. Gmail SMTP Server Settings
   Jika menggunakan Gmail, berikut adalah detil server SMTP :

- SMTP Server: smtp.gmail.com
- SMTP username: Complete Gmail address
- SMTP password: Your Gmail password
- SMTP port (TLS): 587
- SMTP port (SSL): 465
- SMTP TLS/SSL required: yes

SEND AN EMAIL WITH HTML OR RAW TEXT WITH ESP32(ARDUINO IDE)

Kode berikut mengirimkan email melalui Server SMTP dengan HTML atau teks mentah. Untuk tujuan demonstrasi, ESP32 mengirim email satu kali saat boot. Kemudian, Anda bisa mengubah kode dan membuat program Anda sendiri. Jangan mengupload kode dulu, Anda perlu melakukan beberapa modifikasi agar berfungsi.

Anda perlu memasukkan *network credentials* serta mengatur email pengirim, detail Server SMTP, penerima dan pesan.

```
/*
 Rui Santos
 Complete project details at
https://RandomNerdTutorials.com/esp32-send-email-
smtp-server-arduino-ide/
 Permission is hereby granted, free of charge, to
any person obtaining a copy of this software and
associated documentation files.

 The above copyright notice and this permission
notice shall be included in all copies or
substantial portions of the Software.
*/
```

```
#include "ESP32_MailClient.h"

// REPLACE WITH YOUR NETWORK CREDENTIALS
const char* ssid = "REPLACE_WITH_YOUR_SSID";
const char* password =
"REPLACE_WITH_YOUR_PASSWORD";

// To send Email using Gmail use port 465 (SSL)
and SMTP Server smtp.gmail.com
// YOU MUST ENABLE less secure app option
https://myaccount.google.com/lesssecureapps?pli=1
#define emailSenderAccount
"EXAMPLE_EMAIL@gmail.com"
#define emailSenderPassword
"YOUR_EXAMPLE_EMAIL_PASSWORD"
#define emailRecipient
"YOUR_EMAIL_RECIPIENT@EXAMPLE.com"
#define smtpServer "smtp.gmail.com"
#define smtpServerPort 465
#define emailSubject "ESP32 Test"

// The Email Sending data object contains config
and data to send
SMTPData smtpData;

// Callback function to get the Email sending
status
void sendCallback(SendStatus info);

void setup() {
    Serial.begin(115200);
    Serial.println();
    Serial.print("Connecting");
    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(200);
    }

    Serial.println();
    Serial.println("WiFi connected.");
    Serial.println();
    Serial.println("Preparing to send email");
    Serial.println();
```

```
    // Set the SMTP Server Email host, port,
account, and password
    smtpData.setLogin(smtpServer, smtpServerPort,
emailSenderAccount, emailSenderPassword);

    // For library version 1.2.0 and later which
STARTTLS protocol was supported,
    // the STARTTLS will be enabled automatically
when port 587 was used,
    // or enable it manually using setSTARTTLS
function.
    // smtpData.setSTARTTLS(true);

    // Set the sender name and Email
    smtpData.setSender("ESP32",
emailSenderAccount);

    // Set Email priority or importance High,
Normal, Low or 1 to 5 (1 is highest)
    smtpData.setPriority("High");

    // Set the subject
    smtpData.setSubject(emailSubject);

    // Set the message with HTML format
    smtpData.setMessage("<div
style=\"color:#2f4468;\"><h1>Hello
World!</h1><p>- Sent from ESP32 board</p></div>",
true);

    // Set the email message in text format (raw)
    // smtpData.setMessage("Hello World! - Sent
from ESP32 board", false);

    // Add recipients, you can add more than one
recipient
    smtpData.addRecipient(emailRecipient);
    //
smtpData.addRecipient("YOUR_OTHER_RECIPIENT_EMAIL
_ADDRESS@EXAMPLE.com");
    smtpData.setSendCallback(sendCallback);

    // Start sending Email, can be set callback
function to track the status
    if (!MailClient.sendMail(smtpData)) {
```

```
        Serial.println("Error sending Email, " +
MailClient.smtpErrorReason());
    }

    // Clear all data from Email object to free
memory
    smtpData.empty();
}

void loop() {

}

// Callback function to get the Email sending
status
void sendCallback(SendStatus msg) {
    // Print the current status
    Serial.println(msg.info());

    // Do something when complete
    if (msg.success()) {
        Serial.println(" --------------- ");
    }
}
```

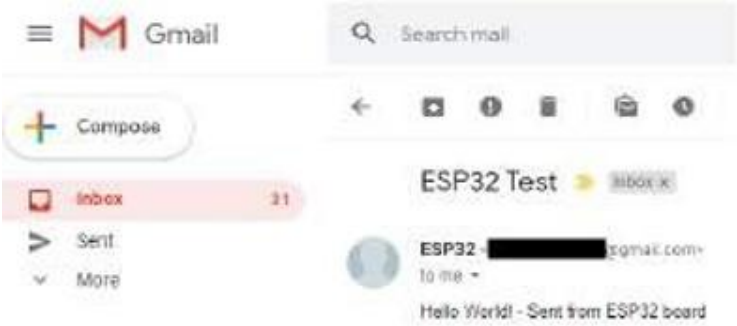
DEMONSTRATION

Upload program ke ESP32 Anda. Setelah mengupload, buka Monitor Serial pada baud rate 115200. Tekan tombol Reset ESP32. Jika semuanya berjalan seperti yang diharapkan, Anda akan mendapatkan pesan serupa di Serial Monitor.

Cek email anda :

Jika anda menyetel opsi untuk mengirim pesan dengan teks HTML, seperti inilah tampilan pesannya:

Jika Anda telah mengaktifkan *raw text messages*, ini adalah email yang akan Anda terima :



SEND ATTACHMENTS VIA EMAIL WITH ESP32 (ARDUINO IDE)

Di bagian ini, akan ditunjukkan cara mengirim lampiran di email Anda yang dikirim oleh ESP32. Praktikum ini akan menunjukkan cara mengirim file gambar atau *.txt*. Ini sangat berguna misalnya untuk mengirim file *.txt* dengan pembacaan sensor dari beberapa jam terakhir, atau untuk mengirim foto.

File yang akan dikirim harus disimpan di **ESP32 SPIFFS** atau di kartu **microSD**. Dalam contoh ini, kami akan menggunakan SPIFFS (tetapi kode yang diberikan dapat diubah untuk menggunakan kartu microSD).

UPLOAD FILES TO SPIFFS

Untuk mengirim file melalui email, ini harus disimpan di ESP32 SPIFFS, atau di kartu microSD. Kami akan mengunggah gambar dan file .txt ke ESP32 SPIFFS menggunakan *ESP32 Filesystem Uploader plugin for Arduino IDE*. Pasang **plugin** Filesystem Uploader in Arduino IDE jika Anda belum memasangnya:

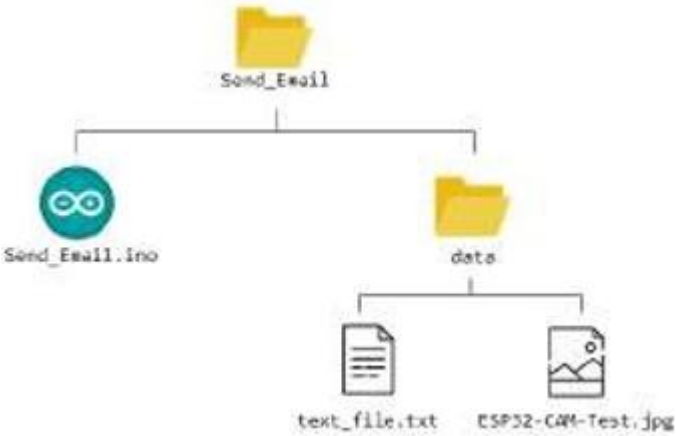
Install ESP32 Filesystem Uploader in Arduino IDE

Buat sketsa Arduino baru dan simpan. Kemudian ke **Sketch> Show Sketch** folder. Di dalam folder sketsa Arduino, buat folder bernama **data**. Pindahkan file jpg dan file .txt ke folder **data** Anda.

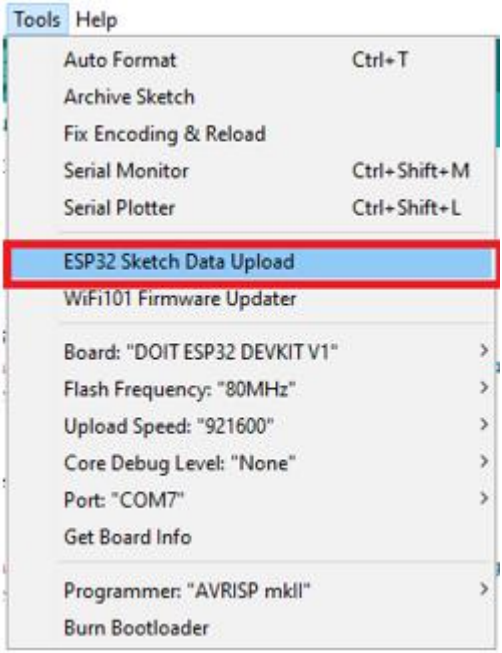
Catatan: dengan kode default file Anda harus dinamai ESP32-CAM-Test.jpg dan text_file.txt atau Anda dapat mengubah kode untuk mengimpor file dengan nama yang berbeda.

Kita akan mengirimkan file-file ini:

Struktur folder Anda akan terlihat seperti berikut :



Setelah memindahkan file ke folder **data**, di Arduino IDE Anda, buka **Tools> ESP32 Data Sketch Upload** dan tunggu file diunggah.



Anda akan mendapatkan pesan sukses di jendela debugging. Jika file berhasil diunggah, lanjutkan ke bagian berikutnya.

CODE

Kode berikut mengirimkan email dengan file .txt dan gambar terlampir. Sebelum mengunggah kode, pastikan Anda memasukkan pengaturan email pengirim serta email penerima Anda.

```
/*
Rui Santos
Complete project details at
https://RandomNerdTutorials.com/esp32-send-
email-smtp-server-arduino-ide/
Permission is hereby granted, free of
charge, to any person obtaining a copy of
this software and associated documentation
files.

The above copyright notice and this
permission notice shall be included in all
copies or substantial portions of the
Software.
*/

#include "ESP32_MailClient.h"
#include "SD.h"
#include "SPIFFS.h"

// REPLACE WITH YOUR NETWORK CREDENTIALS
const char* ssid = "REPLACE_WITH_YOUR_SSID";
const char* password =
"REPLACE_WITH_YOUR_PASSWORD";

// To send Email using Gmail use port 465
(SSL) and SMTP Server smtp.gmail.com
// YOU MUST ENABLE less secure app option
https://myaccount.google.com/lesssecureapps?
pli=1
#define emailSenderAccount
"EXAMPLE_EMAIL@gmail.com"
#define emailSenderPassword
"YOUR_EXAMPLE_EMAIL_PASSWORD"
#define emailRecipient
"YOUR_EMAIL_RECIPIENT@EXAMPLE.com"
#define smtpServer "smtp.gmail.com"
#define smtpServerPort 465
#define emailSubject "ESP32 Test Email with
Attachments"
```

```
// The Email Sending data object contains
config and data to send
SMTPData smtpData;

// Callback function to get the Email
sending status
void sendCallback(SendStatus info);

void setup() {
    Serial.begin(115200);
    Serial.println();

    // Initialize SPIFFS
    if (!SPIFFS.begin(true)) {
        Serial.println("An Error has
occurred while mounting SPIFFS");
        return;
    }

    // Or initialize SD Card (comment the
preceding SPIFFS begin)
    /*
    MailClient.sdBegin(14, 2, 15, 13); //
(SCK, MISO, MOSI, SS)
    if (!SD.begin()) {
        Serial.println("Card Mount Failed");
        return "";
    }
    */

    Serial.print("Connecting");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(200);
    }

    Serial.println();
    Serial.println("WiFi connected.");
    Serial.println();
```

```
    Serial.println("Preparing to send
email");
    Serial.println();

    // Set the SMTP Server Email host, port,
account and password
    smtpData.setLogin(smtpServer,
smtpServerPort, emailSenderAccount,
emailSenderPassword);

    // For library version 1.2.0 and later
which STARTTLS protocol was supported,
    // the STARTTLS will be enabled
automatically when port 587 was used,
    // or enable it manually using
setSTARTTLS function.
    // smtpData.setSTARTTLS(true);

    // Set the sender name and Email
    smtpData.setSender("ESP32",
emailSenderAccount);

    // Set Email priority or importance
High, Normal, Low or 1 to 5 (1 is highest)
    smtpData.setPriority("High");

    // Set the subject
    smtpData.setSubject(emailSubject);

    // Set the message with HTML format
    smtpData.setMessage("<div
style=\"color:#2f4468;\"><h1>Hello
World!</h1><p>- Sent from ESP32
board</p></div>", true);

    // Set the email message in text format
(raw)
    // smtpData.setMessage("Hello World! -
Sent from ESP32 board", false);
```

```
    // Add recipients, you can add more than
one recipient
    smtpData.addRecipient(emailRecipient);
    //
smtpData.addRecipient("YOUR_RECIPIENT_EMAIL_
ADDRESS@example.com");

    // Add attach files from SD card or
SPIFFS
    // Comment the next two lines, if no
SPIFFS files created or SD card connected
    smtpData.addAttachFile("/ESP32-CAM-
Test.jpg", "image/jpg");

smtpData.addAttachFile("/text_file.txt");

    // Add some custom header to message
    // See
https://tools.ietf.org/html/rfc822
    // These header fields can be read from
raw or source of message when it received
    //
smtpData.addCustomMessageHeader("Date: Sat,
10 Aug 2019 21:39:56 -0700 (PDT)");
    // Be careful when set Message-ID, it
should be unique, otherwise message will not
store
    //
smtpData.addCustomMessageHeader("Message-ID:
<abcde.fghij@gmail.com>");

    // Set the storage type to attach files
in your email (SPIFFS or SD Card)

smtpData.setFileStorageType(MailClientStorag
eType::SPIFFS);
    //
smtpData.setFileStorageType(MailClientStorag
eType::SD);

    smtpData.setSendCallback(sendCallback);
```

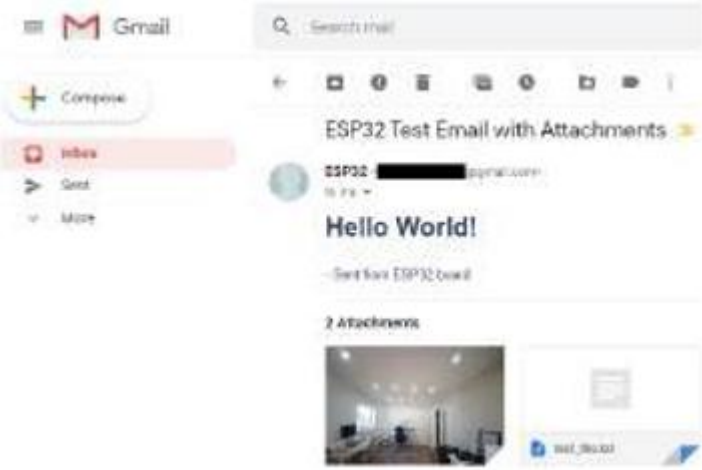
```
    // Start sending Email, can be set
callback function to track the status
    if (!MailClient.sendMail(smtpData)) {
        Serial.println("Error sending Email,
" + MailClient.smtpErrorReason());
    }

    // Clear all data from Email object to
free memory
    smtpData.empty();
}

void loop() {

}

// Callback function to get the Email
sending status
void sendCallback(SendStatus msg) {
    // Print the current status
    Serial.println(msg.info());

    // Do something when complete
    if (msg.success()) {
        Serial.println( --------------- );
    }
}
```

DEMONSTRATION

Setelah mengupload program, buka Serial Monitor pada baud rate 115200 dan tekan tombol EN / RESET on-board. Jika semuanya berjalan lancar, Anda akan mendapatkan pesan serupa di Serial Monitor.

```
Preparing to send email

Connecting to SMTP server...
SMTP server connected, wait for response...
Identification...
Authentication...
Sign in...
Sending Email header...
Sending Email body...
Sending attachments...
/ESP32-CAM-Test.jpg
/text_file.txt
Finalize...
Finished
Email sent successfully
```

Cek Email :



Dalam praktikum ini Anda telah mempelajari cara mengirim email dengan ESP32 menggunakan Server SMTP. Agar metode ini berfungsi, ESP32 harus memiliki akses ke internet.

Jika Anda tidak ingin menggunakan Server SMTP, Anda juga dapat menulis skrip PHP untuk mengirim pemberitahuan email dengan NodeMCU ESP32 atau ESP8266. Anda telah mempelajari cara mengirim email sederhana dengan teks dan lampiran. Saat menggunakan lampiran, ini harus disimpan di kartu SD atau SPIFFS. Hal ini akan mempermudah anda dalam mengirim lampiran/attachment file.

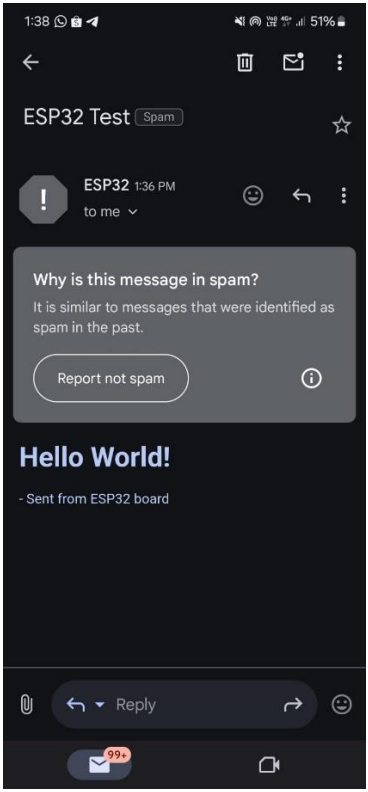
D. Percobaan

1. Kode untuk mengirimkan email melalui Server SMTP dengan HTML

```
#include "ESP32_MailClient.h"

    // Replace with your network credentials
    const char *ssid = "Wifi@id";
const char *password = "tujuhtujuh";

// Email configuration
#define emailSenderAccount "haritammim11@gmail.com"
#define emailSenderPassword "mhqx kqdk dzmn mxni"
#define emailRecipient "mochamadthariq12@gmail.com"
#define smtpServer "smtp.gmail.com"
#define smtpServerPort 465
#define emailSubject "ESP32 Test"

// The Email Sending data object contains config and data to send
SMTPData smtpData;

// Callback function to get the Email sending status
void sendCallback(SendStatus info);

void setup()
{
  Serial.begin(115200);
  Serial.println();

  Serial.print("Connecting to WiFi");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED)
  {
    Serial.print(".");
    delay(200);
  }

  Serial.println();
  Serial.println("Wifi connected.");
  Serial.println("Preparing to send email");

  // Set the SMTP Server Email host, port, account, and password
  smtpData.setLogin(smtpServer, smtpServerPort, emailSenderAccount, emailSenderPassword);

  // Set the sender name and Email
  smtpData.setSender("ESP32", emailSenderAccount);

  // Set Email priority (High, Normal, low or 1 to 5)
  smtpData.setPriority("High");

  // Set the subject
  smtpData.setSubject(emailSubject);

  // Set the message with HTML format
  smtpData.setMessage(
      "<div style=\"color:#2f4468;\"><h1>Hello World!</h1><p>  Sent from ESP32 board</p></div>", true);

  // Add recipients (you can add more than one)
  smtpData.addRecipient(emailRecipient);

  // Set callback to track the status
  smtpData.setSendCallback(sendCallback);

  // Start sending Email
  if (!MailClient.sendMail(smtpData))
  {
    Serial.println("Error sending Email: " + MailClient.smtpErrorReason());
  }

  // Clear all data from Email object to free memory
  smtpData.empty();
}

void loop()
{
  // Empty loop
}

// Callback function to get the Email sending status
void sendCallback(SendStatus msg)
{
  // Print the current status
  Serial.println(msg.info());

  // Do something when complete
  if (msg.success())
  {
    Serial.println("Email sent successfully!");
    Serial.println("----------------");
  }
}
```

Hasil Pengujian :

```
Connecting to WiFi......
WiFi connected.
Preparing to send email
Connecting to SMTP server...
SMTP server connected, wait for response...
Identification...
Authentication...
Sign in...
Sending Email header...
Sending Email body...
Finalize...
Finished
Email sent successfully
Email sent successfully!
----------------
```

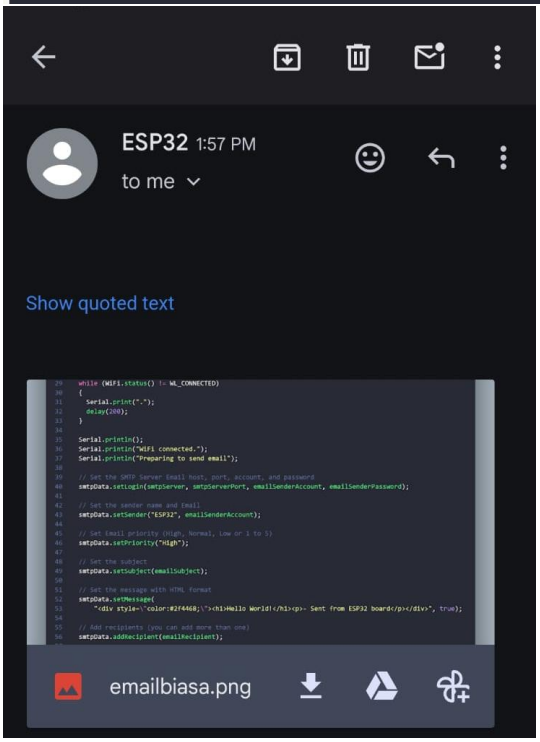


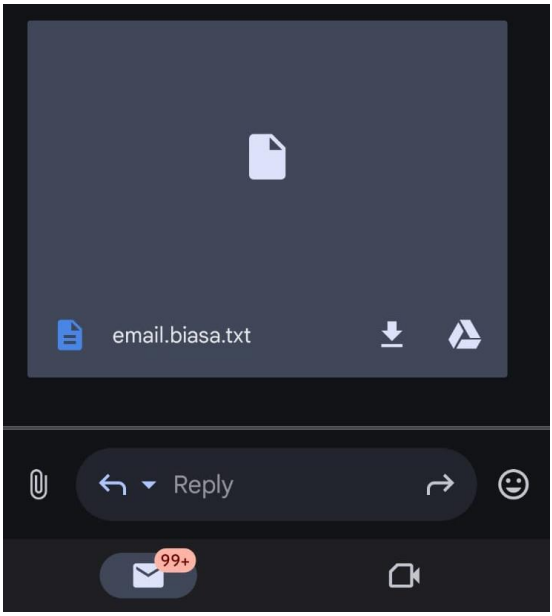
2. Kode untuk mengirimkan email dengan file .txt dan gambar terlampir

```
#include "ESP32 Mail Client.h"
#include "SD.h"
#include "SPIFFS.h"

// Replace with your network credentials
const char *ssid = "Kirim";
const char *password = "rujuntujuh";

// Email configuration
#define emailSenderAccount "muhammad13@gmail.com"
#define emailSenderPassword "rmyx kqvk dvmv mmrl"
#define emailRecipient "mochamadthariq2@gmail.com"
#define smtpServer "smtp.gmail.com"
#define smtpServerPort 465
#define emailSubject "ESP32 Test Email with Attachments"

// The Email Sending data object contains config and data to send
SMTPData smtpData;

// Callback function to get the Email sending status
void sendCallback(SMTPStatus msg);

void setup()
{
  Serial.begin(115200);
  Serial.println();

  // Initialize SPIFFS
  if (!SPIFFS.begin(true))
  {
    Serial.println("An Error has occurred while mounting SPIFFS");
    return;
  }

  // Or Initialize SD card (comment the previous SPIFFS.begin)
  /*
  MailClient.sdBegin(14, 2, 15, 13); // (SCK, MISO, MOSI, SS)
  if(!SD.begin(14))
    Serial.println("Card Mount Failed");
    return "";
  */

  Serial.print("Connecting");

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED)
  {
    Serial.print(".");
    delay(200);
  }

  Serial.println();
  Serial.print("WiFi connected.");
  Serial.print(F());
  Serial.println("Preparing to send email");
  Serial.println();

  // Set the SMTP Server Email host, port, account and password
  smtpData.setLogin(smtpServer, smtpServerPort, emailSenderAccount, emailSenderPassword);

  // For library version 1.2.0 and later which STARTTLS protocol was supported,the STARTTLS will be
  // enabled automatically when port 587 was used, or enable it manually using setSTARTTLS()function.
  // smtpData.setSTARTTLS(true);

  // For debugging
  // Set the session config
  smtpData.setSender("ESP32", emailSenderAccount);

  // Set the priority to High, Normal, Low or 1 to 5 (1 is highest)
  smtpData.setPriority("High");

  // Set the subject
  smtpData.setSubject(emailSubject);

  // Set the message with HTML format
  smtpData.setMessage("<div style='color:#2f4468;'>hello world</br><b>Sent from ESP32 board</b></div>", true);
  // Set the email message in text format (raw)
  // smtpData.setMessage("Hello World! - Sent from ESP32 board", false);

  // Add recipients, you can add more than one recipient
  smtpData.addRecipient(emailRecipient);
  // smtpData.addRecipient("YOUR_RECIPIENT_EMAIL_ADDRESS@example.com");

  // Add attach files from SD card or SPIFFS
  // Comment the next two lines, if no SPIFFS files created or SD card connected
  smtpData.addAttachFile("/mmulibiasa.png", "image/png");
  smtpData.addAttachFile("/email.blank.txt");

  // Add some custom header to message
  // See https://tools.ietf.org/html/rfc822
  // These header fields can be read from raw or source of message when it received.
  // smtpData.addCustomMessageHeader("Date: Sat, 18 Aug 2020 21:30:50 -0700 (PDT)");
  // By configuring when set Message ID, it should be unique. Otherwise message will not sent.
  // smtpData.addCustomMessageHeader("Message-ID: <abcde.fgh@gmail.com>");

  // Set the storage type to attach files in your email (SPIFFS or SD Card)
  smtpData.setFileStorageType(MailClientSDStorageType::SPIFFS);
  // smtpData.setFileStorageType(MailClientSDStorageType::SD);

  smtpData.setSendCallback(sendCallback);

  // Start sending Email, can only set callback function to track the status
  if (!MailClient.sendMail(smtpData))
    Serial.println("Error sending Email, " + MailClient.smtpErrorReason());

  // Clear all data from Email object to free memory
  smtpData.empty();
}

void loop()
{
}

// Callback function to get the Email sending status
void sendCallback(SMTPStatus msg)
{
  // Print the current status
  Serial.println(msg.info());

  // Do something when complete
  if (msg.success())
  {
    Serial.println("----------------");
  }
}
```

Hasil Pengujian :

```
Connecting......
WiFi connected.

Preparing to send email

Connecting to SMTP server...
SMTP server connected, wait for response...
Identification...
Authentication...
Sign in...
Sending Email header...
Sending Email body...
Sending attachments...
/emailbiasa.png
/email.biasa.txt
Finalize...
Finished
Email sent successfully
----------------
```





E. Analisa

Pada praktikum ini, dilakukan pengujian terhadap dua kode yang berfungsi untuk mengirim email menggunakan ESP32. Kode pertama bertujuan untuk mengirim email dalam format HTML atau teks mentah, sedangkan kode kedua memungkinkan pengiriman email dengan lampiran berupa file teks dan gambar. Hasil pengujian menunjukkan bahwa ESP32 berhasil mengirim email dengan kedua metode tersebut.

Proses pengiriman email diawali dengan koneksi ESP32 ke jaringan WiFi menggunakan SSID dan password yang telah dikonfigurasi. Jika

koneksi berhasil, ESP32 akan terhubung ke server SMTP Gmail menggunakan port 465 (SSL) untuk mengautentikasi akun pengirim. Setelah proses autentikasi selesai, ESP32 dapat mengirim email sesuai dengan format yang diatur. Pada kode pertama, email dikirim dalam format teks biasa atau HTML dengan menggunakan fungsi setMessage(), di mana HTML memungkinkan tampilan email lebih menarik dengan elemen-elemen seperti warna, heading, dan paragraf.

Pada kode kedua, selain mengirim teks, ESP32 juga melampirkan file yang disimpan di SPIFFS. Agar proses ini berhasil, sistem file SPIFFS harus dimuat terlebih dahulu. Jika terjadi kesalahan saat membaca file, proses pengiriman lampiran akan gagal. Lampiran yang berhasil dikirim dalam pengujian ini berupa file gambar dan teks yang ditambahkan menggunakan fungsi addAttachFile(). Setelah email berhasil dikirim, ESP32 menampilkan status pengiriman di Serial Monitor untuk memastikan tidak ada kesalahan selama proses berlangsung.

Berdasarkan hasil pengujian, terdapat beberapa aspek yang perlu diperhatikan dalam implementasi sistem ini. Pertama, ESP32 harus memiliki koneksi internet yang stabil agar dapat mengakses server SMTP. Kedua, server SMTP Gmail membutuhkan autentikasi yang mungkin memerlukan konfigurasi tambahan pada akun email pengirim, seperti mengaktifkan fitur Less

Secure Apps atau menggunakan App Password. Ketiga, jika ingin mengirim lampiran, file harus tersedia dan dapat diakses melalui SPIFFS atau kartu microSD. Kesalahan dalam membaca file dapat menyebabkan kegagalan pengiriman lampiran.

F. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa ESP32 mampu mengirim email melalui server SMTP dengan berbagai format, baik dalam bentuk teks biasa, HTML, maupun dengan lampiran file. Proses pengiriman email ini bergantung pada koneksi WiFi yang stabil serta konfigurasi autentikasi yang tepat pada server email. Selain itu, penggunaan sistem file SPIFFS memungkinkan ESP32 untuk menyertakan lampiran dalam email, seperti gambar dan file teks.